

Helix Thready — Installation Guide

Table of contents

1. Prerequisites
2. Installation topology (diagram)
3. Local development install
4. Verifying the local install
5. Hetzner production install
6. Root Admin bootstrap
7. Messenger sign-in & first channel
 - 7.1 Telegram (interactive sign-in)
 - 7.2 Telegram (non-interactive / headless)
 - 7.3 Onboard the first channel
8. Client installs
9. Uninstall / teardown
10. Open items

Rev	Date	Author	Change
1	2026-07-21	swarm (user-guides)	Initial local + Hetzner install procedures
2	2026-07-22	swarm (user-guides, Pass 3)	Depth pass: corrected messenger sign-in to VERIFIED Herald <code>HERALD_MTPROTO_*</code> names (user client) with the <code>HERALD_TGRAM_*</code> reply-path distinction; split the topology diagram explanation into multi-paragraph form; added a first-run verification matrix (§4); linked the new <code>quickstart.md</code> .

This guide takes you from nothing to a running Helix Thready system — first a **local development** stack for evaluation, then the **three-environment Hetzner deployment** (dev/staging/production). It stops where `configuration.md` begins for the deep `.env` reference and where `../deployment/index.md` begins for the full ops runbook.

Zero-version scope. Several subsystems are `[BUILD-NEW]` per the `gap register`. This guide installs the platform and the VERIFIED-production modules; features that depend on a scaffold (Max reading, generic HTTP downloads, OCR) are called out where you'd expect them.

Table of contents

1. Prerequisites
2. Installation topology (diagram)
3. Local development install
4. Verifying the local install
5. Hetzner production install
6. Root Admin bootstrap
7. Messenger sign-in & first channel
8. Client installs (CLI/TUI/Desktop/Mobile)
9. Uninstall / teardown
10. Open items

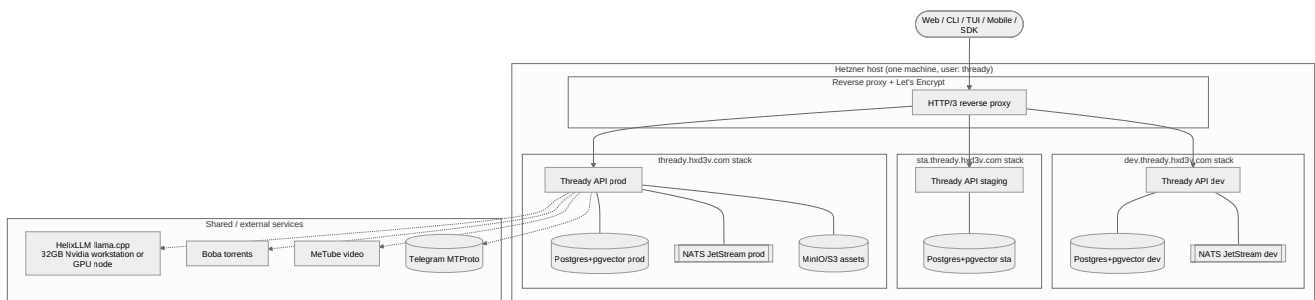
1. Prerequisites

Requirement	Version / spec	Provenance
OS	Linux (x86-64/arm64); rootless-capable kernel	<code>[CONSTITUTION §11.4.76]</code>
Container runtime	rootless Podman + <code>podman compose</code> (no Docker daemon, no root)	<code>[CONSTITUTION §11.4.76/161]</code>
Go	1.26.x (to build from source / CLI)	<code>[CONSTITUTION silent → project]</code>
PostgreSQL	16+ with the pgvector extension	<code>[IN-HOUSE: vectordb]</code>
HelixLLM host	llama.cpp server reachable; GPU recommended (workstation 32 GB Nvidia)	<code>[IN-HOUSE: HelixLLM]</code>
Node.js	20+ (only to build the Angular portal)	<code>[IN-HOUSE: design_system]</code>
Disk	dev: ~20 GB; prod: object storage for 50 TB+ assets (MinIO/S3)	<code>[OPERATOR: scale]</code>

Production hardware baseline `[DEFAULT – adjustable]` (Q5): ≥16 vCPU, ≥64 GB RAM, NVMe for Postgres/pgvector, large object storage (MinIO/S3), plus a GPU node (or the workstation) for HelixLLM.

Credentials. You need Telegram `api_id` / `api_hash` (from my.telegram.org) and the operator account phone. These live only in `.env` / `$HOME/api_keys.sh` / the private repo — never in a public repo [CONSTITUTION §11.4.10] .

2. Installation topology (diagram)



Rendered PNG/SVG exported via Docs Chain (§11.4.65). Source: [diagrams/install-topology.mmd](#).

Explanation (for readers/models that cannot see the diagram). A single Hetzner host runs three **fully separated** container stacks — `dev.`, `sta.` and `thready.` (production). The separation is the load-bearing property of this topology: each stack carries its own Postgres+pgvector database and, for the busier environments, its own NATS JetStream bus, and the production stack additionally has a MinIO/S3 object tier sized for the 50 TB+ asset scale. Because the databases and buses are per-stack, a schema migration, a data-wipe, or a runaway job in `dev.` cannot reach into `sta.` or production — the three environments never share data or ports.

A single HTTP/3 reverse proxy sits in front of all three. It terminates TLS with one Let’s Encrypt certificate per subdomain and routes each hostname to its isolated stack. This is why the client URL you use (`dev.` / `sta.` / bare) deterministically selects an environment, and why there is no way to “accidentally” cross environments from a client — routing happens once, at the proxy, by hostname.

Outside the host sit the **shared services** the platform delegates to rather than re-implements: HelixLLM (llama.cpp) for embeddings/research/vision — typically the operator’s 32 GB Nvidia workstation or a dedicated GPU node; Boba for torrents; MeTube for video downloads; and the Telegram MTPROTO endpoint that Herald’s user client (`HERALD_MTPROTO_*`) connects to. These are drawn as dotted edges from production because they are reached over the network and are not part of any single stack — several can be shared across environments or pointed at per-environment instances via configuration.

All clients (Web, CLI, TUI, Mobile, SDK) reach the platform **only** through the reverse proxy; they never touch a stack directly. That single ingress is what lets RBAC, rate limiting, and TLS be enforced uniformly regardless of surface. This is the topology `installation.md` provisions; the full compose/rollback/backup detail lives in `../deployment/index.md`.

3. Local development install

The fastest path to a running system for evaluation. Uses SQLite, the in-process bus, and a local llama.cpp embedder.

Step 1 — Clone with submodules.

```
git clone git@github.com:HelixDevelopment/helix_thready.git
cd helix_thready
git submodule update --init --recursive # pulls docs/private if you have access
```

Step 2 — Start HelixLLM (real embedder). [GAP: 1] The default HelixLLM embedder is a non-semantic hash stub. Start llama.cpp with a real embedding model and point Thready at it:

```
# example: llama.cpp server exposing an OpenAI-compatible /v1
llama-server --embedding --host 0.0.0.0 --port 8080 \
  -m ./models/jina-embeddings-v2-base-code.gguf
export HELIX_EMBEDDING_PROVIDER=llama # NEVER leave this on 'hash' for search
```

Step 3 — Create your .env. Copy the template and fill it (see [configuration.md §3](#)):

```
cp .env.example .env
chmod 600 .env
$EDITOR .env
```

Step 4 — Bring up datastores (rootless Podman).

```
# Postgres + pgvector for the vector store (SQLite handles relational in dev)
podman compose -f deploy/compose.dev.yml up -d postgres
```

Step 5 — Run migrations and start the API.

```
./thready db migrate up # digital.vasic.database migration.Runner
./thready serve # REST /v1 + WS/SSE on THREADY_HTTP_ADDR
```

If a required variable is missing the process **fails loudly and lists it** — fix .env and retry.

4. Verifying the local install

```
# 1. Liveness / readiness (observability/pkg/health)
curl -k https://localhost:8443/health/ready

# 2. API version handshake
curl -k https://localhost:8443/v1/meta/version

# 3. Embedding sanity — MUST be a real model, not the hash stub [GAP: 1]
./thready doctor embeddings
# ✓ provider=llama model=jina-embeddings-v2-base-code dim=1024
# ✗ provider=hash → ABORT: non-semantic embedder blocked for search
```

`thready doctor` (see [cli-reference.md §5.8](#)) runs the full readiness matrix: DB reachable, pgvector present, embedder real, NATS reachable (prod), storage writable, and messenger session valid. A green `doctor` is the gate before onboarding a channel.

First-run verification matrix. Work top to bottom; each row is independently checkable, and the “Expected” column is the exact signal a healthy install gives. A failure at any row is diagnosed in the [troubleshooting](#) section named in the last column.

#	Check	Command	Expected	If it fails
1	Process boots	<code>./thready serve</code>	Logs <code>listening on 0.0.0.0:8443</code> , no <code>ABORT</code>	Service won't start
2	Readiness	<code>curl -k https://localhost:8443/health/ready</code>	<code>200 {"status":"ready"}</code>	Service won't start
3	Version handshake	<code>curl -k https://localhost:8443/v1/meta/version</code>	<code>200 {"version":...}</code>	Service won't start
4	DB + pgvector	<code>./thready doctor (db row)</code>	<code>✓ db reachable</code> , <code>✓ pgvector present</code>	Service won't start
5	Real embedder	<code>./thready doctor embeddings</code>	<code>✓ provider=llama ... dim=1024</code>	Search irrelevant
6	Config resolves	<code>./thready config validate</code>	<code>✓ all required vars present</code>	Config not taking effect
7	Messenger session	<code>./thready doctor messenger</code>	<code>✓ telegram session valid</code> ; <code>max = not implemented</code>	Can't sign in
8	Storage writable	<code>./thready doctor storage</code>	<code>✓ store writable</code> , signed URLs OK	Downloads

Rows 1–6 must pass before onboarding a channel; rows 7–8 must pass before you expect assets. Row 5 is the single most important gate — a green embedder is what makes semantic search real rather than garbage [GAP: 1].

5. Hetzner production install

Full runbook: [../deployment/index.md](#). Summary here.

Step 1 — Provision the `thready` user (root, one time). Root SSH provisions the unprivileged `thready` user with home `/home/thready`; all three environments run under it (final request \$14.2).

```
ssh -p "$SSH_PORT_HXDEV" root@"$ADDRESS_HXDEV"
adduser --home /home/thready thready
loginctl enable-linger thready           # rootless Podman services survive logout
```

Step 2 — Deploy the three stacks. As `thready`, run the deployment script; it stands up the isolated dev/staging/prod compose stacks, requests Let's Encrypt certs per subdomain, and wires the reverse proxy:

```
su - thready
git clone git@github.com:HelixDevelopment/helix_thready.git && cd helix_thready
./deploy/deploy.sh production           # also: development | staging
```

Step 3 — Secrets. Production secrets come from the private-repo mount + `$HOME/api_keys.sh`, never the public repo. The deploy script sources them at runtime and validates the required set before starting each service.

Step 4 — Verify each environment.

```
for e in dev sta ""; do
  curl -sf "https://${e:+$e.}thready.hxd3v.com/health/ready" && echo " ✓ ${e:-prod}"
done
```

6. Root Admin bootstrap

Exactly **one** Root Admin exists and is created at deploy time, owner-only (final request \$21.7). The bootstrap is idempotent and refuses to run twice.

```
# Runs only if no root admin exists; credentials from env/secrets, never printed
./thready admin bootstrap \
  --email "$THREADY_ROOT_EMAIL" \
  --from-secret THREADY_ROOT_PASSWORD
#   ✓ Root Admin created. MFA enrolment required at first login (TOTP).
```

After bootstrap, the Root Admin logs into the Web portal (or CLI) and **must enrol TOTP MFA** (mandatory for admin tiers, `[DEFAULT — adjustable]` Q9). Next steps for the Root Admin — creating Accounts, setting global retention, and white-label branding — are in the `root-admin-guide.md`.

7. Messenger sign-in & first channel

`[GAP: 3]` **VERIFIED status before you start.** Telegram reading uses the `gotd/td` MTPROTO **user** client that is being promoted from Herald's `qaherald` QA harness to a first-class channel (`[BUILD-NEW]` P0). **Max reading is not available yet** — the adapter (Bot API + Go OneMe user-WebSocket port) is `[BUILD-NEW]` P0; only reserved env vars exist. Plan around Telegram for the zero version.

7.1 Telegram (interactive sign-in)

```
./thready messenger login telegram
#   → sends a login code to the Telegram USER account (HERALD_MTPROTO_PHONE)
#   Enter code: 12345
#   Enter 2FA password (if enabled): ***** # HERALD_MTPROTO_PASSWORD
#   ✓ Session saved to HERALD_MTPROTO_SESSION_FILE (default ~/.config/herald/mtproto.session)
```

This uses Herald's **MTPROTO user client** (`HERALD_MTPROTO_APP_ID` / `APP_HASH` / `PHONE`), the only transport that can read a channel's full history. The Bot-API path (`HERALD_TGRAM_BOT_TOKEN`) is used separately for posting status replies — a bot token alone can never backfill history.

7.2 Telegram (non-interactive / headless)

For headless deploy, set `THREADY_MESSENGER_SIGNIN_MODE=noninteractive` and provide the `HERALD_MTPROTO_*` variables in the environment (VERIFIED names — see `configuration.md` §9); the first run establishes the session file, later runs reuse it. Persist `HERALD_MTPROTO_SESSION_FILE` on an encrypted, writable path so a container restart does not force a re-login (which would need a fresh SMS code).

7.3 Onboard the first channel

```
# Test fixture from the final request Appendix A
./thready channel add --messenger telegram --invite "https://t.me/+622y04wzy_Yz0TA0"
#   ✓ Channel joined. Auto-recognition queued. Poll interval: 5m.
./thready channel list
```

Once a channel is added, Thready assembles each **complete post** (root + the full organic reply chain, excluding the system's own replies), stores it, and begins processing per hashtag/content type. What happens next — categories, status replies, search — is the [end-user-manual.md](#).

8. Client installs

Client	Install	Guide
CLI	<code>go install github.com/HelixDevelopment/helix_thready/cmd/thready@latest</code> (or download a release binary)	cli-reference.md
TUI	Same binary: <code>thready tui</code>	tui-usage.md
Web portal	Served by the deploy at <code>https://thready.hxd3v.com</code> ; nothing to install	web-portal-guide.md
Desktop	Tauri 2 bundle per OS (<code>.AppImage</code> / <code>.dmg</code> / <code>.msi</code>) from releases	web-portal-guide.md \$Desktop
Mobile	Firebase App Distribution (dev/staging) or store builds	mobile-guide.md

Desktop/Mobile caveat. The Tauri desktop shell wraps the Angular portal (org standard). Mobile native clients depend on `Security-KMP`, whose mobile secure storage is currently an **in-memory stub** [GAP: 7] — see [mobile-guide.md §3](#) before installing on a real device.

9. Uninstall / teardown

```
# Local
podman compose -f deploy/compose.dev.yml down -v
rm -rf ./data

# Hetzner (per environment, preserves the other two)
./deploy/teardown.sh staging          # dev | staging | production
```

Teardown is environment-scoped and never touches a sibling environment (the isolation guarantee). Backups are retained per the backup policy ([configuration.md §14](#)).

10. Open items

- `[OPEN: inst-1]` Max messenger sign-in cannot be documented end-to-end until the `[BUILD-NEW]` Max adapter exists `[GAP: 3]`. Workable item: **ATM — Max adapter (Bot API + OneMe WebSocket)**, then add a \$7.4 Max sign-in flow here.
 - `[OPEN: inst-2]` The `deploy/deploy.sh` / `compose.*.yaml` filenames are placeholders pending the deployment area's canonical script names. Workable item: **ATM — reconcile deploy script names with deployment/**.
 - `[OPEN: inst-3]` `thready doctor` sub-checks assume the diagnostics command surface in [cli-reference.md](#); keep the two in sync once the CLI is implemented.
-

Made with love ♥ by Helix Development.